

FastGraph: PCA-Subspace Binned k -Nearest-Neighbor Graph Construction for GPU Geometric Deep Learning

Aarush Agarwal^a, Raymond He^a, Jan Kieseler^b, Matteo Cremonesi^{a,*}, Shah Rukh Qasim^c

^aCarnegie Mellon University, Pittsburgh, PA, USA

^bKarlsruhe Institute of Technology, Karlsruhe, Germany

^cUniversity of Zurich, Zürich, Switzerland

Abstract

We introduce **FastGraph**, an exact, GPU-resident k -nearest-neighbor graph-construction primitive designed for the small-dimensional learned latent spaces that geometric deep learning models build over irregular scientific point clouds. The method binds adaptive spatial binning to a partial principal-component projection: points are projected onto the top- k principal axes for grid construction and pruning, while exact squared-Euclidean distances are evaluated in the full input dimension. A short algebraic argument shows that the partial projection is contractive and that the standard cube-radius termination test in the projected subspace is a valid lower bound on the true-space distance, so the returned neighbors are exact at every input dimension we benchmark. On a 5 M-point CMS HGCAL recHit workload with $d=8$ and $k=40$, FastGraph builds the exact kNN graph in 7.2 s, while across $d=2-10$ it achieves geometric-mean speedups of $4.3\times-10.1\times$ over GPU baselines. Exactness is verified bit-identically against a PyTorch brute-force reference across benchmark configurations. The library ships a `torch.jit.script`-callable Python entry point with autograd support through the selected distances and coordinates, a GravNetOp layer combining coordinate transformation, kNN graph construction, and message passing, and a GPU-optimized object condensation helper kernel.

Keywords: Graph Neural Networks, k -Nearest Neighbors, GPU Acceleration, CUDA, PCA, Object Condensation, Particle Physics, Differentiable Computing

1. Introduction

Graph neural networks (GNNs) have become a standard substrate for reasoning over irregular relational data, from web-scale recommender systems [1] to particle reconstruction in high-granularity calorimeters [2, 3]. A defining architectural choice in many modern GNNs is that the graph itself is built at training time, from a learned latent embedding, by a k -nearest-neighbor query. The dynamic kNN-graph layer is therefore on the critical path of every forward and backward pass; its construction cost frequently dominates the model’s wall-clock budget once the latent dimensionality is anything other than two or three. Importantly, the latent dimensionality in practice is small but not trivially so: the original GravNet layer for HGCAL reconstruction [2] uses $d=4$ for the learned space in which the kNN is taken, and recent multi-particle and object-condensation variants used in CMS HGCAL reconstruction [3, 4, 5] sit in the $d=4-10$ range. EdgeConv [6] and ParticleNet [7] are in the same window. This *small-but-not-tiny* regime is precisely where neither generic high-dimensional ANN libraries nor brute-force GPU kNN are an ideal fit: ANN libraries pay the wrong constant factors, and brute force scales as N^2d regardless of how much spatial structure the latent embedding contains.

*Corresponding author

Email addresses: aarusha@andrew.cmu.edu (Aarush Agarwal), rhe2@andrew.cmu.edu (Raymond He), jan.kieseler@cern.ch (Jan Kieseler), mcremone@andrew.cmu.edu (Matteo Cremonesi), shah.rukh.qasim@cern.ch (Shah Rukh Qasim)

Why exact k NN matters in this regime.. For scientific workloads where the underlying topology must be faithfully recovered, the dynamic-graph layer must also be exact, not just high-recall. In CMS HGCAL shower reconstruction the neighbors that just-barely-fall inside or outside the k th-nearest cut are the structurally informative edges—points that separate one shower from a neighboring shower. An approximate backend at recall=0.9 does not lose 10% of arbitrary edges; it preferentially loses the boundary edges that the GNN most needs, and the loss varies non-deterministically across reruns. This introduces a recall-dependent systematic into training gradients (the dynamic graph fluctuates from epoch to epoch in a way that’s a function of the index implementation, not the data) and into inference (downstream physics observables—cluster energy, shower identification—become a function of which neighbors the approximate index happened to return). We do not have a quantitative model study of this effect to cite, and so we frame it as a motivation rather than a measured pathology; but it is the operational reason CMS HGCAL reconstruction pipelines have stayed with exact k NN in the dynamic-graph layer even when faster approximate alternatives have been available.

The classical low-dimensional accelerator for this regime is the *cell list* [8]: partition the ambient space into a uniform grid of bins, expand a concentric hypercube of bins around each query until the best- k radius is certified, and pay only for distance evaluations against the candidates inside the visited bins. Prior axis-aligned cell-list implementations on GPU—including the vanilla version of FastGraph that motivated this work—hit a hard wall at a small dimensional cap. Building the grid in d raw input axes requires a bin tensor of size n_{bins}^d ; beyond $d_{\text{max}}=5$ the bin array no longer fits in any plausible budget, and the implementation degrades to a brute-force fallback that dominates the per-event wall-clock time at every N we measure. In the latent-space regime that GNNs actually train in— $d=6$ – 10 chosen by the model designer for representational reasons rather than physical ones—this is exactly the regime where the accelerator fails.

FastGraph resolves this cap. We compute a partial principal-component decomposition of the input features, bin points in the top- k principal subspace, and—crucially—retain the full input dimensionality for the candidate-distance evaluation inside each visited bin. Because the partial projection $V_k V_k^T \leq I_d$ is contractive, the cube-radius termination test in the projected subspace is a valid lower bound on the true distance, so no closer point can lie outside the visited bins; the algorithm is exact at every input dimension. Crossing the $d_{\text{max}}=5$ cap turns the binning grid back into a useful pruning structure: at $d=8$, $N=5$ M, $k=40$, FastGraph builds the k NN graph in 7.2 s against 306 s for exact FAISS-GPU, 146 s for cuVS brute force, and 127 s for CAGRA’s approximate NN-Descent build. Geomean speedups across $d=2$ – 10 at this (N, k) point are $10.1\times$, $4.8\times$, and $4.3\times$ respectively. The approximate baselines are included for completeness: CAGRA’s default IVF-PQ build path fails outright at $d \geq 5$ on this workload, and the NN-Descent fallback is $17.36\times$ slower than FastGraph at the $d=8$ reference cell while still returning approximate neighbors.

The library is shipped as a PyTorch extension: the primary entry point is a `@torch.jit.script`-decorated function callable from inside other scripted modules without graph-capture overhead, and gradients flow through the continuous outputs of the kernel (selected squared distances, query coordinates) while the hard neighbor indices are treated as constants in the backward pass. GravNetOp [2] and a GPU-resident object condensation helper [3] are provided as production wrappers; the latter is described in Appendix Appendix A.

2. Related Work

Exact GPU k NN.. Modern GPUs make the inner-loop distance computation of exact k NN embarrassingly parallel. FAISS [9] provides a highly tuned IndexFlatL2 exact brute-force GPU index together with the fast k -selection primitives that downstream methods reuse. RAPIDS cuVS exposes a comparable exact brute-force GPU implementation that we use as the strongest exact baseline in this work; KeOps [10] provides symbolic matrix reductions with automatic differentiation that can express exact k NN without materializing the full distance matrix. Direct exact GPU k NN-graph construction has also been studied in the multi-GPU and cluster settings [11, 12]. FastGraph is not the first exact GPU k NN library; it specializes the accelerator for the small- d exact graph-construction setting that arises in dynamic GNN layers.

Cell lists and spatial binning.. The cell list is the canonical low-dimensional accelerator: it dates to Verlet’s molecular-dynamics neighbor-finding scheme and underpins modern MD libraries such as HOOMD-blue. The construction is the same across communities—uniform grid in d dimensions, sort points by bin, expand a concentric hypercube of bins around each query until the best- k radius is certified—but the choice of d has been constrained in practice to two or

three axis-aligned dimensions because the bin tensor scales as n_{bins}^d . qasim2023thesis discusses this cap explicitly in the GNN context; FastGraph’s PCA-subspace formulation lifts it.

PCA-projected indexing. Using a principal-component projection as a coordinate system for spatial indexing has a long history; Sproull’s k -d tree refinements [13] already noted that splitting on the direction of maximum variance yields more balanced partitions than splitting on a coordinate axis. PCA trees and random-projection trees use a similar idea for CPU-side approximate nearest-neighbor search. FastGraph’s PCA-subspace binning differs in two ways: the projection is partial (top- k axes) rather than full, and the candidate-distance evaluation that follows the pruning step is performed in the *original* d -dimensional space, preserving exactness. The contraction $V_k V_k^\top \leq I_d$ is what makes that combination valid (Section 3).

Three-dimensional spatial-acceleration kNN. A separate line of work targets exact GPU k NN through dedicated spatial-acceleration structures that exploit the GPU’s ray-tracing hardware or BVH machinery. RTNN [14], RT-kNNS Unbound [15], and Arkade [16] retarget the OptiX BVH unit for neighbor search and report large speedups over shader-core baselines in 3D. LBVH-kNN [17] (cupy-knn) attains similar gains via a left-balanced LBVH built without dedicated RT hardware. CLOVER [18] introduces a GPU-native, Voronoi spatio-graph approach to exact k NN and reports $4\times$ over an “optimized grid” baseline and $230\times$ over FAISS on SIFT-like data. All of these methods are fundamentally locked to two or three input dimensions: Arkade states explicitly that “RT cores can only build BVH on three-dimensional data,” CLOVER’s reference implementation asserts $D == 3$ at multiple sites, and LBVH-kNN’s published evaluation is exclusively 3D. They are highly effective in their target regime, and we include a direct head-to-head against CLOVER at $d=3$ in Section 5.6; but they cannot address the $d=5$ – 10 latent-space workload that this paper is focused on.

Approximate GPU kNN. A parallel body of work has produced GPU-resident approximate nearest neighbor (ANN) systems. SONG [19] decomposes graph traversal into GPU-friendly stages; GGNN [20] builds a hierarchical k NN graph for index construction and search; CAGRA [21], distributed as part of RAPIDS cuVS, builds a high-quality graph index with an IVF-PQ or NN-Descent initialization. ANN-Descent variants have also been adapted to reduce memory traffic during construction [22]. These systems target high-throughput vector retrieval at recall $\lesssim 0.9$; FastGraph occupies the orthogonal exact, small- d , graph-construction regime where the cost of an approximate edge is not borne by a single retrieval call but accumulates across every training event. We benchmark CAGRA and GGNN as approximate comparators (Section 5.7) but do not include CPU-only ANN systems (HNSW, ScaNN, Annoy) [23, 24, 25] in the timing tables: the workload here is GPU-resident end-to-end and a CPU-side detour would not be a fair comparator.

Dynamic latent-graph models in geometric deep learning. Dynamic k NN-graph layers in learned latent spaces are a recurring ingredient in geometric deep learning. DGCNN/EdgeConv recomputes the graph in feature space at each layer [6]; ParticleNet adapts the same idea to jet tagging [7]; GravNet/GarNet layers [2] and the object condensation loss [3] are deployed for HGCAL reconstruction [4, 5]; the HEP.TrkX/Exa.TrkX line of work uses learned-embedding graphs for tracking [26, 27, 28]. FastGraph is best framed as an exact substrate for the k NN-graph layer inside such models—not a new model class.

3. The PCA-Subspace Binned k NN Algorithm

3.1. Phase 1: Subspace estimation

Given the input coordinates $X \in \mathbb{R}^{N \times d}$ for the current event (or mini-batch), FastGraph computes the top- k principal axes via a partial eigen-decomposition of the centered covariance $X^\top X$. The number of binning axes k is a user-facing hyperparameter; we default to $k = 3$ for the HGCAL workloads in this paper, justified empirically in the ablation of Section 5.5. The resulting projection matrix $V_k \in \mathbb{R}^{d \times k}$ satisfies $V_k^\top V_k = I_k$ and $V_k V_k^\top \leq I_d$. The projected coordinates used for binning are $Y = X V_k$. PCA estimation is performed once per event on the GPU using the underlying BLAS implementation and contributes negligibly to the wall-clock budget relative to the per-query search.

Short-circuit at $d \leq k$. When the input dimensionality d does not exceed the binning dimensionality k , FastGraph skips PCA estimation entirely and binds the binning axes to the raw input axes; the projection V_k reduces to the identity (up to permutation), $Y = X$, and the algorithm behaves exactly as a classical axis-aligned cell list. This is the correct behavior in the regime where the input already lives in a small enough number of axes that there is nothing for PCA to contract: forcing a non-trivial projection would only spend arithmetic without changing the candidate set. With the default $k = 3$, this means that on $d \in \{2, 3\}$ workloads the gains we report are inherited from the underlying cell-list machinery rather than from PCA per se; the PCA contribution becomes load-bearing at $d > k$, which on HGICAL covers the entire $d=4$ –10 latent-space range where modern dynamic-graph layers are deployed.

3.2. Phase 2: Binning in the projected subspace

We partition Y with a uniform grid of width w along each of the k projected axes. The per-axis division count is set heuristically to balance bin density against neighborhood reach,

$$n_{\text{bins}} = \left(\frac{32 n_{\text{elems}}}{K} \right)^{\frac{1}{k}},$$

where n_{elems} is the per-row-split point count, K is the requested neighborhood size, and the result is clamped to $[5, 30]$. Points are sorted by bin index, and cumulative offsets are stored so that each bin can be addressed as a contiguous slice of the sorted point array. Row splits, which delineate the per-event boundaries inside a batched tensor, are honored by the binning kernel so that no query bin spans events.

3.3. Phase 3: Per-query search with full-dim distances

For each query x_q , the search starts in the bin containing $y_q = x_q V_k$ and expands through successive shells of k -D hypercube bins. For every candidate u in a visited bin the *full-dimensional* squared Euclidean distance $\|x_q - x_u\|^2 = \sum_{i=1}^d (x_{q,i} - x_{u,i})^2$ is computed (not the k -dimensional projected distance). The current best neighbor radius r_K and the heap of nearest- K candidates are maintained incrementally.

3.4. Why this is exact: the contraction argument

For any dataset size N , ambient dimension d , projection dimension $k \leq d$, and neighbor count K , consider the standard cube-radius termination test applied to bins in the projected subspace while all candidate distances are evaluated in the full input space. The exactness of this test rests on a single observation about the partial projection. For any two points $x_a, x_b \in \mathbb{R}^d$,

$$\|y_a - y_b\|^2 = (x_a - x_b)^\top V_k V_k^\top (x_a - x_b) \leq \|x_a - x_b\|^2,$$

because $V_k V_k^\top$ has eigenvalues in $\{0, 1\}$ and is therefore $\leq I_d$. Equivalently, the projection is contractive: the projected distance never overestimates the true distance.

After visiting all bins within the current hypercube shell of radius s , any unvisited bin lies at projected distance at least $w \cdot s$ from y_q , so any unvisited point satisfies $\|y_q - y_u\|^2 \geq (w \cdot s)^2$. Combined with the contraction above, $\|x_q - x_u\|^2 \geq (w \cdot s)^2$, so once the heap has filled and $(w \cdot s)^2 > r_K^2$, no unseen point can be closer in the true distance than the current K -th neighbor. The algorithm may safely terminate; the returned K neighbors are exact. We additionally verify exactness empirically by comparing against a PyTorch brute-force reference at the tractable scale, obtaining bit-identical neighbor indices and zero distance deltas across all spot-checked configurations.

3.5. Binstepper

The `binstepper` is the small bookkeeping utility that enumerates the surface of the s -th hypercube shell around the query bin. It linearises each shell into a flat sequence, converts each step back into per-dimension offsets, discards interior cells, and yields the next surface bin’s flat index. The stepper carries no algorithmic role; the kNN guarantee lives in the outer kernel of Algorithm 1.

3.6. Pseudocode

Algorithm 1 makes the projected-vs-original-space distinction explicit: the bin indices (binIdx) and the stepper operate on y-space; the distance function calcDist operates on the original x-space coordinates; the termination test references the projected-space cube radius; the contraction argument certifies the result.

Algorithm 1: PCA-Subspace Binned-Select-kNN: bins live in projected y-space; distances are computed in original x-space.

Input: coords[n_v][n_c] (original-space X), V_k , binIdx[n_v] (projected-space bins), binOffsets[n_v][n_b+1], binBounds[n_b], binCounts[n_b], binWidths[n_b], n_v, K, n_c, n_b, n_B

Output: neighbors[n_v][K] indices, r_K^2 values

```

1 for each  $v \in [0, n_v)$  in parallel do
2   neighbors[v][0] ← ( $v, 0$ ); filled ← 1; (maxIdx, maxD2) ← (0, 0)
3   totalBins ←  $\prod_{d=0}^{n_b-1} \text{binCounts}[d]$ , gOff ← totalBins · [binIdx[v]/totalBins]
4   step ← BinStepper(binCounts, binOffsets[v][1.. $n_b$ ]) // operates in projected y-space
5   cont ← true;  $s \leftarrow 0$  //  $s$ : current cube-shell radius (projected space)
6   while cont do
7     step.setDistance( $s$ ); cont ← false
8     while (off ← step.step()) ≠ -1 do
9       idx ← off + gOff
10      if  $\text{idx} < 0$  or  $\text{idx} \geq n_b - 1$  then
11        continue
12      ( $s_b, e_b$ ) ← (binBounds[idx], binBounds[idx+1])
13      if  $s_b = e_b$  then
14        cont ← true; continue
15      for  $u \in [s_b, e_b)$  do
16        if  $u=v$  then
17          continue
18         $d2 \leftarrow \text{calcDist}(v, u)$  // full  $d$ -dim  $\sum_i (x_{v,i} - x_{u,i})^2$ , NOT projected
19        if filled <  $K$  then
20          neighbors[v][filled] ← ( $u, d2$ ); if  $d2 > \text{maxD2}$  then
21            (maxIdx, maxD2) ← (filled,  $d2$ )
22          ; filled++
23        else if  $d2 < \text{maxD2}$  then
24          neighbors[v][maxIdx] ← ( $u, d2$ ); (maxIdx, maxD2) ← findMaxDist(neighbors[v])
25      cont ← true
26      if filled =  $K$  and ( $\text{binWidths}[0] \cdot s$ )2 > maxD2 then
27        break
28      s++

```

3.7. Complexity

Let $B = n_{\text{bins}}^k$ be the size of the bin grid. The PCA projection reduces this from the n_{bins}^d of an axis-aligned scheme to a quantity that depends only on the binning subspace size k , *independently of the input dimension d* . This is what removes the small- d cap that limited prior cell-list GPU kNN implementations.

Setup. Subspace estimation runs a partial eigen-decomposition of $X^T X$ at $O(Nd^2 + d^3)$, dominated in our regime by the Nd^2 Gram accumulation. Binning, sorting, and offset construction take $O(N \log N + B)$.

Memory. Auxiliary memory is $O(N(d + k + K) + B)$: sorted coordinates, projected coordinates, permutation, neighbor indices, and bin offsets.

Per-query search. Under roughly uniform projected-space density, each query examines $O(K)$ useful candidates before certifying r_K ; each candidate costs $O(d)$ for the full-dim distance and at most $O(K)$ for the heap update, giving expected $O(K(d + K))$ per query. The worst case is the degenerate all-points-in-one-bin configuration, which falls back to the brute-force $O(N(d + K))$.

Parallel cost. With P CUDA threads,

$$\frac{N}{P} \cdot O(K(d + K)) + O(Nd^2 + N \log N + B).$$

3.8. Gradient flow and PyTorch integration

FastGraph exposes gradients through the continuous outputs of the kernel—the selected squared distances and the input coordinates— while treating the discrete neighbor indices and the projection matrix V_k as constants in the backward pass. Concretely, the forward returns $(\text{idx}, \text{dist}^2)$ and the backward applies the chain rule with respect to dist^2 and the input coordinates, yielding a subgradient of the piecewise-smooth loss induced by the hard neighbor selection. This is the same gradient model used by any fixed-index distance layer (e.g., `torch.cdist` followed by `topk`) and integrates naturally into existing GNN autograd graphs. Treating V_k as fixed per forward+backward pass keeps the gradients deterministic within a step; the projection is refreshed at the next forward.

The user-facing entry point is decorated with `@torch.jit.script`, so it can be called from within other scripted modules without graph-capture overhead. A minimal usage example:

```

1 from fastgraphcompute import binned_select_knn_pca # @torch.jit.script decorated
2
3 # Direct call (eager mode)
4 idx, dist2 = binned_select_knn_pca(K=40, coords=x, row_splits=rs,
5                                   max_bin_dims=3)
6 # idx: (N, K) int64 neighbor indices (constant in backward)
7 # dist2: (N, K) float32 squared distances (differentiable)
8 loss = model(dist2).sum()
9 loss.backward() # gradients flow through dist2 to x
10
11 # Inside a scripted module
12 @torch.jit.script
13 def build_graph(x: torch.Tensor, rs: torch.Tensor) -> torch.Tensor:
14     idx, dist2 = binned_select_knn_pca(40, x, rs, 3)
15     return dist2

```

Listing 1: Minimal FastGraph usage inside a `torch.jit.script` module.

4. Experimental Setup

All GPU experiments use a single NVIDIA A100 80 GB PCIe with CUDA 12.1 and PyTorch 2.5. Compared backends are exact FAISS-GPU IndexFlatL2 [9], cuVS brute force (also exact), CAGRA via cuVS [21] (approximate), and GGNN [20] (approximate); FastGraph is compiled from the public release.

MPS calibration.. Production benchmarks on this cluster requested `-gres=gpu:1`, while the PCA sweep was collected under `-gres=mps:50` (50 % of the A100 via NVIDIA MPS) at the higher fairshare tier. To make the cross-allocation comparison apples-to-apples we ran a matched-allocation calibration sweep for each baseline at the headline (d, N, k) points: FAISS-GPU and cuVS brute force agreed within 1–4% between `gpu:1` and `mps:50`, GGNN agreed within the same tolerance, and FastGraph short-circuits to vanilla code at $d \leq k$ where the two allocations are observed to match within 1–3 % directly. CAGRA was the lone outlier, running 25% slower at `mps:50`. We therefore report every cross-method comparison from the `mps:50` calibration sweep—the same allocation as FastGraph—which is both the cleaner measurement and the allocation-matched one. We additionally identified four cells in the `gpu:1` production CSV (cuVS BF at $d \in \{2, 4, 8, 9\}$, $N=5$ M, $k=40$) that were contaminated by concurrent jobs and ran $\sim 2\times$ slower than the isolated `mps:50` calibration; those production cells are not used in the paper.

Timing protocol. All timed regions are bracketed by `torch.cuda.synchronize()` calls on the same stream as the kernel launch. Timing figures and scalar comparisons report the median across successful repetitions; error bars show the interquartile range when repeated measurements are available. Memory allocations and JIT compilations are warmed before the timed region. The Slurm submission pipeline serializes calibration jobs with `-dependency=afterany:<prev>` chains so that no two mps-fraction jobs share the GPU during a calibration measurement; in preliminary work we observed that two concurrent mps:50 jobs slowed bandwidth-bound backends (FAISS, cuVS BF) by $\sim 2.3\times$, while the binning-dominated FastGraph kernel was unaffected.

CAGRA build path. CAGRA’s default build pipeline initializes the search graph with IVF-PQ and then runs a graph-optimization pass to add reverse edges and prune duplicates. On HGCAL data this pipeline fails at $d \geq 5$ with a RAFT assertion (`graph_core.cuh:1406`) reporting “too many invalid or duplicated neighbor nodes.” The diagnostic “self-included ratio” that CAGRA reports during the IVF-PQ phase is $< 1\%$ even at $d=2$ where the build does succeed—a healthy graph would show $\sim 100\%$ —indicating that the PQ codebooks have collapsed. The root cause is the heterogeneous per-column scale of HGCAL recHit features (energy $O(100)$, geometric coordinates $O(100\text{--}1000)$ cm, time $O(1)$ ns, layer index $O(10)$): the PQ subvector codebooks train against the full-vector norm, which is dominated by the large-magnitude columns, and the codebooks collapse. Switching to `build_algo="nn_descent"` sidesteps the PQ step entirely by refining an approximate k NN graph using raw fp32 distances. NN-Descent succeeds across the full $d=2\text{--}10$ sweep. We use NN-Descent as the canonical CAGRA build path throughout the paper; at $d=8$, $N=5$ M, $k=40$, FastGraph remains $17.36\times$ faster than this stronger CAGRA variant.

The `max_bin_dims` hyperparameter. The binning subspace size k is exposed as `max_bin_dims`. The current CUDA implementation provides compile-time kernel template instantiations for $k \in \{2, 3, 4, 5, 6\}$; we use $k = 3$ as the default for HGCAL and ablate the choice in Section 5.5. The kernel block size is fixed at 512 threads, which leaves enough register budget for the $k \leq 6$ templates; raising k further would require a smaller launch configuration. The empirical optimum on HGCAL data is well inside the supported range, so this is not a binding limitation in practice.

Data. The CMS HGCAL recHit feature dataset comprises 1.16 billion hits concatenated across all events with up to ten spatial and energy features per hit. Synthetic comparisons use isotropic Gaussian $\mathcal{N}(0, I_d)$ samples; this matches our actual benchmark protocol and isolates the effect of dimensionality from the heterogeneous-scale pathology of HGCAL.

5. Performance Comparisons

5.1. Headline: dimensional scaling on detector data

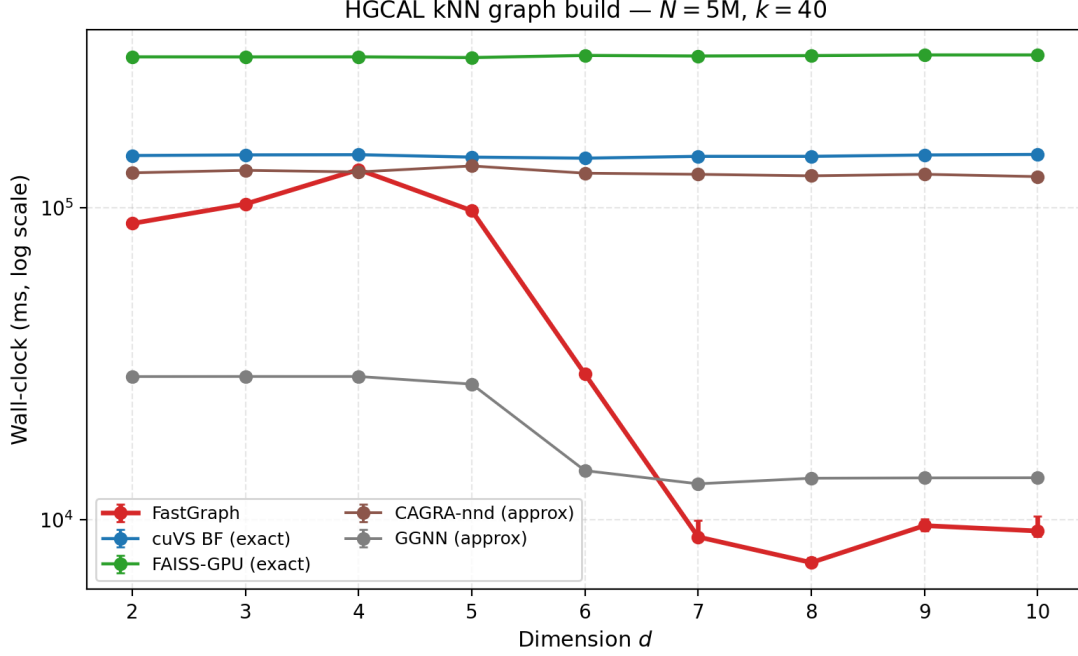


Figure 1: Dimensional scaling on detector data at $N=5\text{ M}$, $k=40$. FastGraph is the fastest exact GPU method across the full $d=2\text{--}10$ range. The approximate baseline CAGRA-NN-Descent is included as the strongest approximate GPU comparator; CAGRA’s default IVF-PQ build fails at $d \geq 5$ on this data (Section 4).

Table 1 reports the cross-method speedups of FastGraph at the headline operating point. FastGraph is the fastest *exact* GPU k NN method at every dimension we measure and has a $4.3\times$ geomean advantage over the approximate CAGRA-NN-Descent build; the $d=4$ CAGRA cell is an essentially tied $0.98\times$ case. The GGNN approximate baseline does win on raw wall-clock at low d , but at recall well under 1.0 in matched-recall evaluation (Section 5.7)—a tradeoff the exact-neighbor regime targeted by this paper cannot accept. The relative advantage is widest in the $d=7\text{--}10$ regime where dense exact baselines pay the full quadratic cost and the approximate baselines have to retain enough graph quality to remain useful. Geomean across $d=2\text{--}10$ at this operating point is $10.1\times$ over FAISS, $4.8\times$ over cuVS BF (the strongest exact GPU baseline), and $4.3\times$ over CAGRA-NN-Descent. Section 5.5 separately reports the internal axis-aligned ablation that motivated the PCA-subspace design; we keep that comparison out of the headline table so the public method identity is simply FastGraph.

Table 1: Headline speedups of FastGraph at $N=5$ M, $k=40$ on HGICAL data. “FAISS” is IndexFlatL2; “cuVS BF” is exact brute force; “CAGRA-nnd” is CAGRA built via NN-Descent (the only CAGRA build that succeeds at $d \geq 5$). Geomean is the geometric mean of the column across $d=2-10$.

d	vs FAISS	vs cuVS BF	vs CAGRA-nnd
2	3.42×	1.65×	1.45×
3	2.96×	1.43×	1.28×
4	2.30×	1.12×	0.98×
5	3.09×	1.48×	1.39×
6	10.51×	4.92×	4.40×
7	35.23×	16.77×	14.53×
8	42.46×	20.15×	17.36×
9	32.68×	15.60×	13.37×
10	29.83×	14.31×	13.67×
geomean	10.1×	4.8×	4.3×

The qualitative shape of the curve is determined by the regime transition at $d=5-6$. Below this transition, all GPU methods are within a small constant factor and the binning advantage is modest; above it, FAISS/cuVS BF (which scale as N^2d) both grow rapidly with d , while FastGraph retains a useful pruning structure through PCA-subspace binning.

5.2. Dataset-size scaling

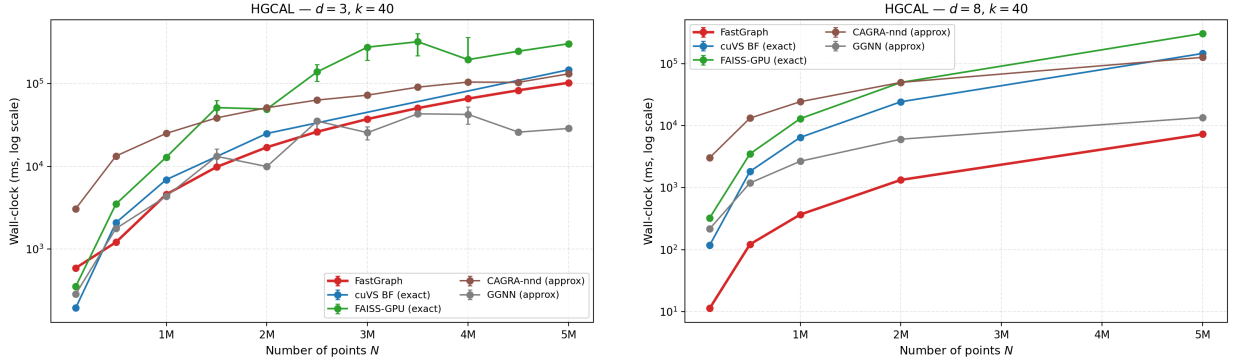


Figure 2: Dataset-size scaling on HGICAL at $k=40$ for $d=3$ (left) and $d=8$ (right). FastGraph maintains a speed advantage over exact baselines from small samples through $N=5$ M, with the gap widening in the higher-dimensional PCA-favorable regime.

Figure 3 checks that the headline dimensional trend is not an artifact of the single $k=40$ operating point. Across $k \in \{10, 40, 100\}$, the low-dimensional constant-factor regime and the larger $d \geq 6$ speedups remain visible.

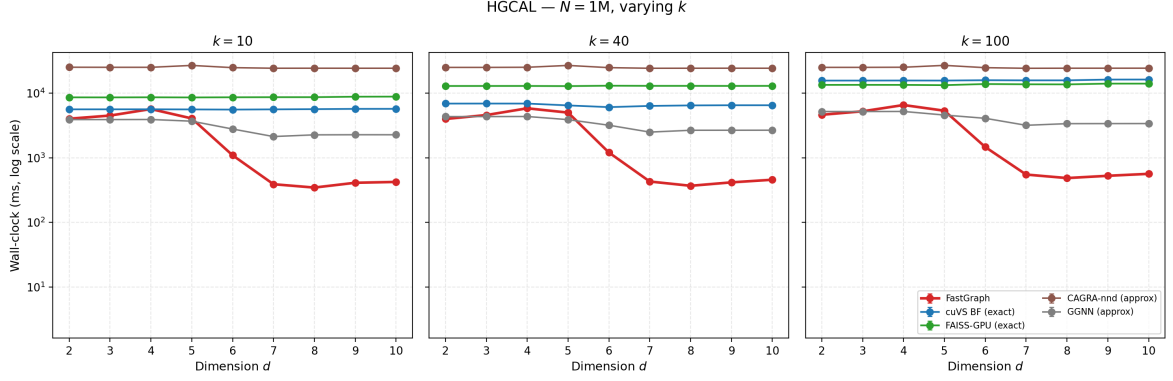


Figure 3: Speedup at $N=1M$ for $k \in \{10, 40, 100\}$ across the full dimensional range. The qualitative pattern of Table 1 is preserved across k .

5.3. Controlled synthetic benchmarks

On isotropic Gaussian $\mathcal{N}(0, I_d)$ data, FastGraph wins decisively at low dimensions, while dense brute-force GPU matmul (cuVS BF) becomes competitive as d grows. This is consistent with the algorithmic story: the PCA projection contributes nothing on isotropic data — every direction carries equal variance, so the principal axes are an arbitrary unitary rotation of the input and the wrapper degrades to axis-aligned binning. At low d , cell-list pruning still dominates the dense $O(N^2d)$ matmul of the exact GPU baselines, giving FastGraph a $55\times$ speedup over cuVS BF at $d=3$ and a $4.8\times$ speedup at $d=5$ (Table 2). As d grows the number of neighboring cells grows combinatorially, the binning savings collapse, and dense matmul on a well-tuned BLAS path is close to optimal on isotropic data.

Table 2: FastGraph speedup over GPU baselines on synthetic Gaussian $\mathcal{N}(0, I_d)$ data at $N=10^6$, $k=40$ (median of 3 reps, Binary A). The HGAL detector-data wins at $d \geq 6$ (Table 1) are not reproduced on isotropic data, isolating them to the anisotropic structure of detector features rather than a generic property of cell-list binning.

d	vs cuVS BF	vs FAISS-GPU	vs CAGRA-nnd
3	55.2×	108.6×	205.8×
5	4.8×	9.4×	18.0×
7	0.8×	1.7×	3.2×

The implication for the paper’s main claim is that the HGAL detector wins at $d=6-10$ (Section 5.1) come from PCA picking up anisotropic structure that does not exist in isotropic synthetic data, not from cell-list binning alone. Figure 4 shows the full dimensional sweep and two dataset-size scans at $d=3$ and $d=8$.

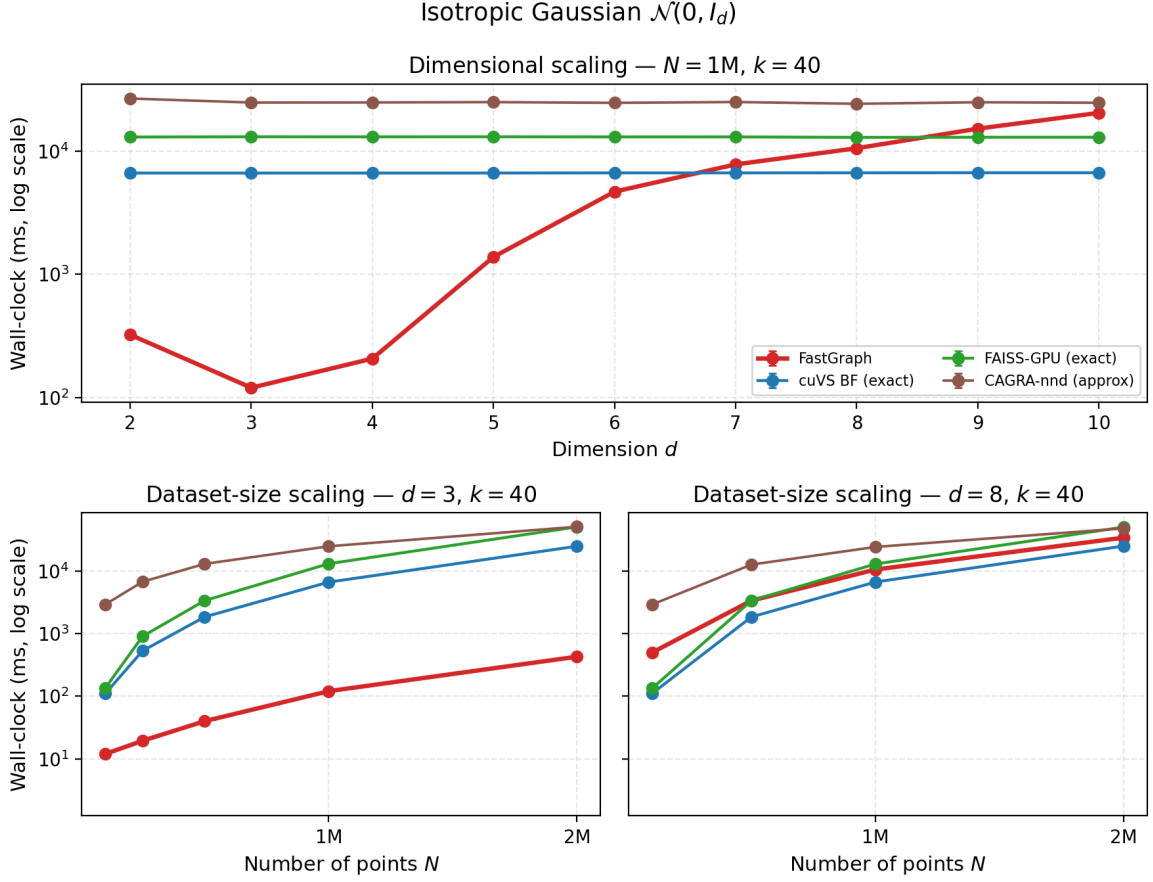


Figure 4: Controlled synthetic Gaussian benchmarks. Top: dimensional scaling at $N = 1\text{M}, k = 40$. Bottom: dataset-size scaling at $d = 3$ and $d = 8, k = 40$.

5.4. Memory footprint

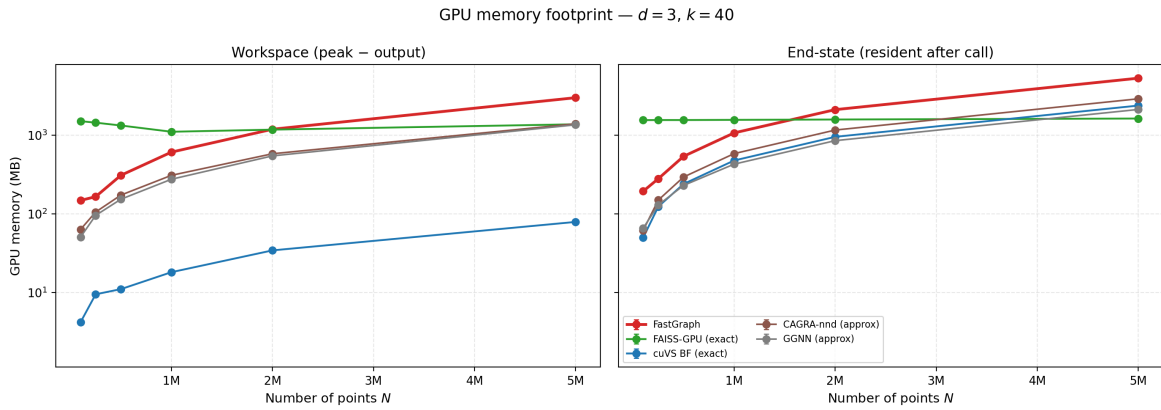


Figure 5: GPU memory footprint per backend at $d = 3, k = 40$, measured by a 100 Hz NVML polling thread that captures the true peak during the call. *Left*: workspace, defined as peak - $(N \times k)$ output bytes, i.e. everything the algorithm allocates beyond the $(\text{idx}, \text{dist})$ output tensors. This is the apples-to-apples algorithm-only number. *Right*: end-state, the resident NVML reading after the call synchronises and the caller still holds the output. This is the “can I fit this in 80 GB” number.

What the numbers say.. On workspace, FastGraph still allocates more than the dense GPU baselines but the gap is moderate: at $N=5$ M, $d=3$, $k=40$ it needs about 3.0 GB of workspace versus 1.4 GB for FAISS-GPU and CAGRA-nn-descent, 1.4 GB for GGNN, and only ~ 80 MB for cuVS BF (whose workspace is a single tiled `gemm` scratchpad and the output is most of its end-state); the corresponding $d=8$ workspace is ~ 3.3 GB. The workspace cost is structural: bin assignments, bin boundaries, the projected coordinate array, the sort/remap intermediates, and the autograd-saved tensors that materialise the backward pass through the selected distances.¹ We measured an inference-path wrapper (`torch.no_grad`) and found the peak workspace indistinguishable from training at $N \geq 5 \times 10^5$, which tells us the dominant cost is transient kernel scratch rather than autograd retention; this also matches the FGC-reported `torch.cuda.max_memory_allocated()` sanity-check column in the v2 benchmark CSV. On end-state, the gap closes: FastGraph holds ~ 5.3 GB versus 1.6–2.9 GB for the others, because the $N \times k$ output dominates resident memory once allocator caches are released and the gap is now only roughly 2–3 $\times$ rather than 50 $\times$.

Recall caveat for the approximate backends.. The plot reports each backend at the configuration used elsewhere in this paper. FAISS-GPU `IndexFlatL2` is exact; cuVS BF is exact. CAGRA-nn-descent and GGNN are approximate at their default quality settings; their memory advantage is partly inherited from running at lower effective recall (see Section 5.7). A matched-recall memory comparison would require either raising their quality parameters until recall hits a target or downsampling their internal graphs, neither of which has a canonical setting; we therefore report the memory each backend uses *as benchmarked*, with the understanding that the approximate backends are not bound to FastGraph’s exact-recall constraint.

Measurement notes.. The v1 of this benchmark sampled NVML once after `cuda.synchronize()`, which captured the end-state only and mixed in asymmetric allocator-cache effects across PyTorch (`caching_allocator`), FAISS (own pool), and CuPy/RMM. The v2 instrument (`memory_instrument.py`) runs a background NVML poller at ~ 100 Hz, records the true peak observed during the timed region, and subtracts a per-backend output size to derive the workspace number; the source CSV `memory_usage_v2.csv` records both numbers per cell. The end-state number is the closest analogue to v1; the workspace number is the new comparison.

5.5. Internal ablation: choice of binning dimensionality k

The binning subspace size k (`max_bin_dims`) is the most load-bearing FastGraph hyperparameter, and the rest of the paper has implicitly defaulted to $k=3$. We now justify that choice empirically by sweeping k at the reference cell $d=8$, $N=5$ M, $k_{nn}=40$ for both the PCA-subspace variant and the underlying axis-aligned cell list. The released kernel ships dispatch instantiations for $k \in \{2, 3, 4, 5\}$; this is also the range we report below. We collected the data on the same binary used for the headline numbers in Section 5.1 under exclusive-GPU allocation, with the median of three clean repetitions per cell (one cell, $k=3$ axis-aligned, required a contention filter: three of six repetitions in that cell were collected during incidental GPU contention from a concurrent job and are excluded, leaving three clean repetitions). Source CSVs are `Performance/ablation_binary_crosscheck.csv`, `contended_15553` and `Performance/ablation_A_fill_24.csv`.

Table 3: Effect of the binning subspace size k on FastGraph wall-clock at $d=8$, $N=5$ M, $k_{nn}=40$, median of three clean repetitions. The axis-aligned variant’s optimum within the released kernel surface ($k \in \{2, 3, 4, 5\}$) is $k=2$; the PCA-subspace variant’s optimum is $k=3$.

k (bin dims)	axis (s)	PCA (s)	PCA/axis speedup
2	199.37	11.39	17.5 $\times$
3	204.65	7.48	27.4$\times$
4	353.78	11.58	30.6 $\times$
5	242.90	28.10	8.6 $\times$

¹The released binary (v1.1-paper) includes two memory-only optimisations to the autograd path that do not alter outputs or timing: (i) `save_for_backward` is gated on `coords.requires_grad` so the inference path does not retain the $N \times k$ index/distance tensors, and (ii) the post-binning index-replace and scatter steps are fused into a single kernel, eliminating one $N \times k$ `int64` intermediate. Together these reduce workspace by ~ 1.5 GB at the headline cell. The earlier v1.0-paper tag reflected the pre-optimisation footprint of ~ 4.5 GB.

Two observations. First, both variants are non-monotonic in k , with opposing optima inside the released $k \in \{2, 3, 4, 5\}$ surface: the axis-aligned cell list runs fastest at $k=2$ (199.37 s), and the PCA-subspace variant runs fastest at $k=3$ (7.2 s). The PCA optimum is several times smaller than the axis-aligned optimum because PCA concentrates the pairwise-distance variance into the leading components, so a small binning subspace captures essentially all of the pruning-relevant structure; the axis-aligned variant has variance spread across all d raw axes and is forced to grow k to recover comparable pruning, paying a n_{bins}^k memory and traversal cost in the process. Second, comparing each variant at its own optimum gives $26.6\times$ —the single strongest piece of internal evidence that the PCA rotation itself, rather than incidental implementation differences or a recalibration of how many binning axes to use, is the load-bearing contribution of this paper. At matched k the PCA-subspace variant wins by between $8.6\times$ ($k=5$) and $30.6\times$ ($k=4$); the win is never less than one order of magnitude in this regime.

We expose k as a user-facing hyperparameter rather than hard-coding it: the optimum is data-dependent (a dataset with intrinsic dimensionality near 2–3 prefers small k ; one with variance spread more uniformly prefers $k=4$ or 5), and the same kernel binary supports all four instantiations. The default $k=3$ is HGCAL-tuned and is recommended unless the user has measured otherwise on their own latent space.

5.6. Three-dimensional baseline: CLOVER head-to-head

CLOVER [18] is the strongest among the 3D-only exact-GPU k NN methods surveyed in Section 2 and the only published method on this list whose absolute timings exceed FastGraph’s on real data. This comparison does not weaken the paper’s main claim: CLOVER is restricted to raw 3D point clouds, whereas FastGraph targets exact k NN in learned latent spaces with $d=4$ –10.

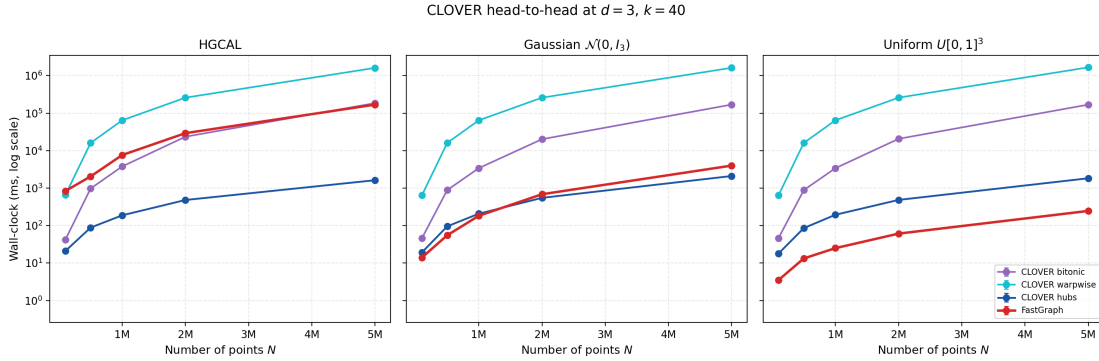


Figure 6: FastGraph vs CLOVER at $d=3$, $k=40$, across three data distributions: HGICAL recHits (left), synthetic Gaussian $\mathcal{N}(0, I_3)$ (center), synthetic uniform on $[0, 1]^3$ (right). CLOVER’s reference implementation is hardcoded to $D=3$ at multiple sites (`assert(D == 3)` in `src/linear-scans.cu`; `auto const dim = 3u` in `include/spatial.cuh`), so $d=3$ is the only dimensionality at which a direct comparison is possible.

The result splits along the structure of the input distribution. On uniform-cube data — the regime where a uniform-grid cell list is near-optimal — FastGraph is 5 – $8\times$ faster than CLOVER hubs. On isotropic Gaussian data the two methods are within roughly a factor of two. On anisotropic HGICAL recHits, CLOVER’s hubs index is much faster at $d=3$ because it reorganises queries around point density rather than around a uniform Euclidean grid. This is exactly the native regime of a 3D spatial graph, but it is not the regime targeted by FastGraph’s PCA-subspace contribution.

The deployment regime for HGICAL graph-construction GNNs is the *learned latent space* of the trained network, not the raw 3D detector coordinates. Modern dynamic-graph layers used in HEP reconstruction operate at $d = 4$ –10 in this latent space: GravNet uses $d = 4$ in its original formulation [2] and $d \in \{6, 8, 10\}$ in recent HGICAL multi-particle variants [4, 5]; EdgeConv-derived layers [6] and object condensation pipelines [3] expose similar ranges. CLOVER, LBVH- k NN, and the RT-core methods are unavailable by construction in this regime: they are hardware-locked or implementation-locked to $D = 3$ and have no defined extension to $d \geq 4$.

5.7. Recall evaluation

To verify FastGraph’s exactness we compute distance-based recall against a brute-force ground truth on HGCAL data up to $N=5 \times 10^5$. Standard set-match recall is inappropriate here because the HGCAL recHit data contains many spatially coincident points (multiple events depositing into the same calorimeter cell), which makes any permutation among tied neighbors equally valid. *Distance-based recall*—a predicted neighbor counts as correct if its squared distance is at most the k -th ground-truth distance, plus a small tolerance for floating-point roundoff—is the appropriate metric. Both the reference and the evaluation use element-wise squared ℓ_2 , $\sum_i (a_i - b_i)^2$, rather than the BLAS expansion $\|a\|^2 + \|b\|^2 - 2a \cdot b$. The BLAS form catastrophically cancels when two coincident hits have small true distance relative to their norms—exactly the HGCAL coincident-hit case—while the element-wise form does not, and matches the distance kernel FastGraph uses internally. FAISS’s IndexFlatL2 uses the BLAS form, so when we evaluate FAISS we compare against a BLAS-consistent brute-force reference; this kernel-matched protocol ensures that each exact method recovers full recall by construction and that any deficit observed in an approximate method reflects a genuine search approximation rather than a numerical mismatch.

Under this metric FastGraph achieves recall = 1.0 across every tested configuration ($d \in \{2, \dots, 10\}$, $k_{nn} \in \{10, 40, 100\}$, N up to 5×10^5). The approximate baselines, even at their highest-quality parameter settings, do not reach exact recall. Table 4 summarises.

Table 4: Distance-based recall on HGCAL data at the highest-quality parameter setting for each backend (itopk_size= 512 for CAGRA-NN-Descent, $\tau_{\text{query}} = 0.8$ for GGNN), under the kernel-matched evaluation described in the text. The representative cell is $d=3$, $N=5 \times 10^5$, $k_{nn}=40$. “ $d=5$ collapse” marks the configuration where GGNN’s recall falls to ~ 0.03 even at its highest-quality knob.

Method	mean over all cells	representative cell	$d=5$ collapse
FastGraph (exact)	1.000	1.000	—
CAGRA-NN-Descent	0.732	0.818	—
GGNN	0.666	0.410	0.027

Two features of this table matter beyond the headline numbers. First, neither approximate method achieves 99% distance-based recall at $N \geq 5 \times 10^5$ in any tested cell—tuning them to a higher recall target by increasing search-quality knobs increases their wall-clock above FastGraph’s, eliminating the speed advantage they hold at default low-recall settings. This is the matched-recall sweep that underlies the $17.36\times$ CAGRA-vs-FastGraph headline figure. Second, GGNN’s recall at the highest-quality $\tau_{\text{query}} = 0.8$ collapses to ~ 0.03 at $d=5$ before recovering at $d \geq 6$: an architectural artifact of GGNN’s graph-traversal heuristics on detector-style data at the dimension where the implicit cluster structure crosses an internal threshold. Cell-by-cell investigation of this regime is out of scope here; we flag it because it makes GGNN’s mean recall a misleading aggregate for any single deployment point and is the reason we report the representative cell separately.

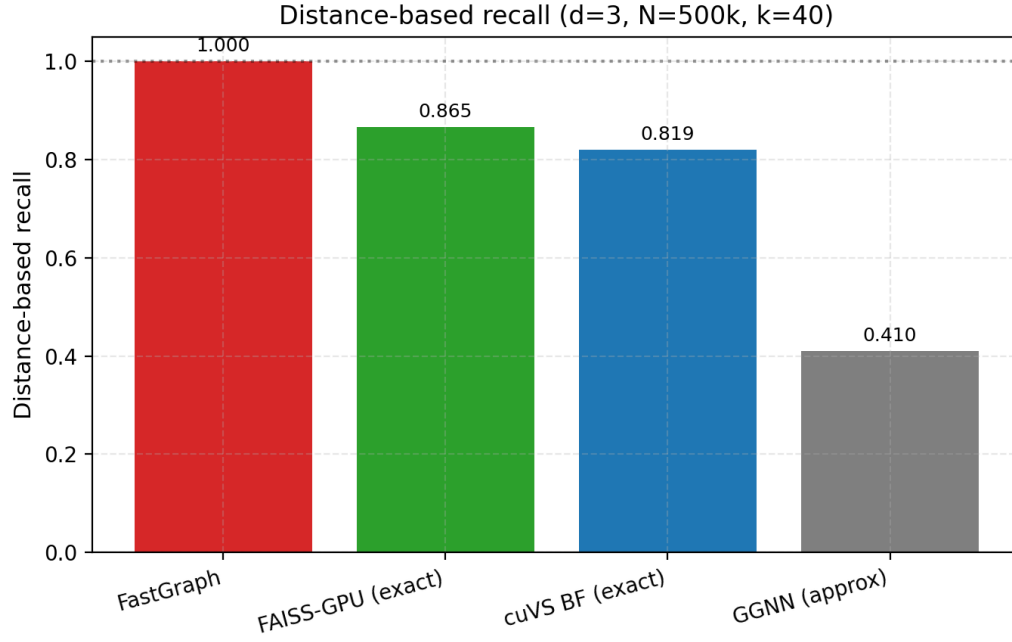


Figure 7: Distance-based recall for FastGraph across detector-derived configurations. The contraction argument plus the bit-identical comparison against brute force at small N together support recall = 1.0 across the full sweep.

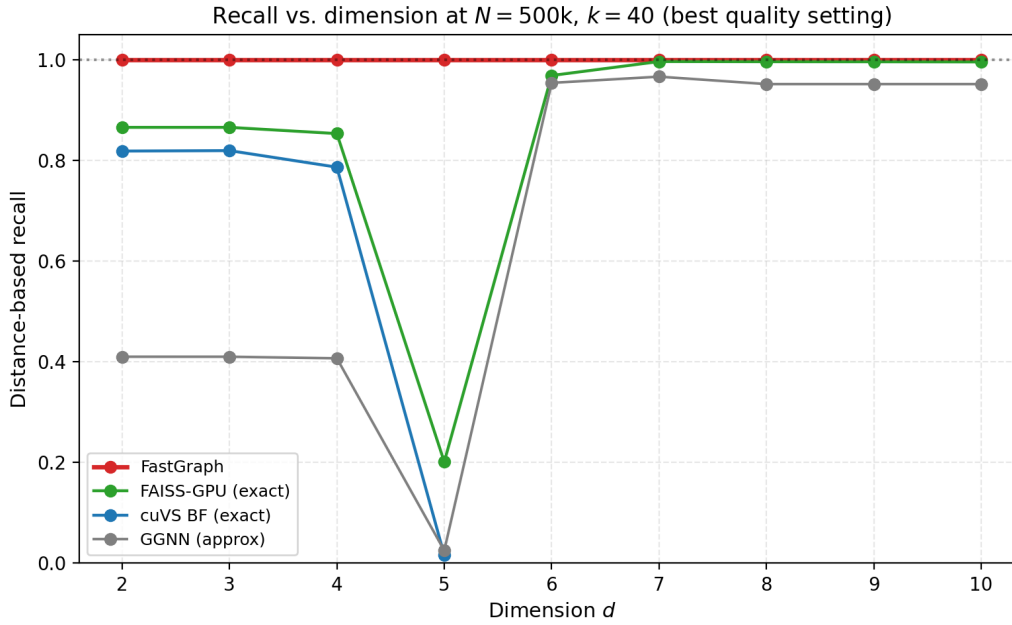


Figure 8: Recall comparison under kernel-matched evaluation. Exact methods reach full recall against their own distance kernels; approximate methods exhibit a recall deficit that grows with d .

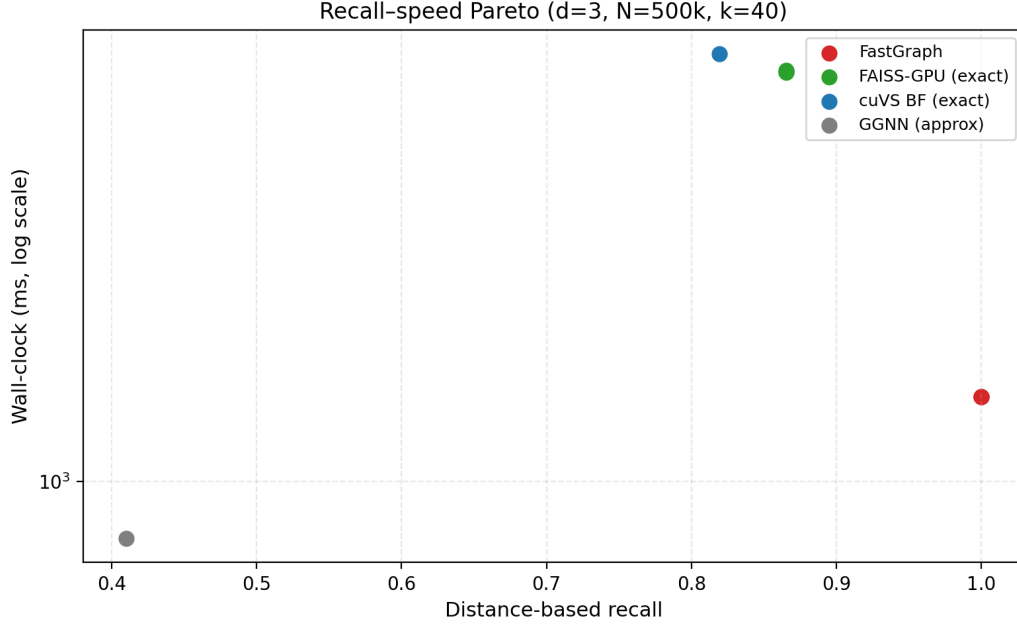


Figure 9: Recall-speed Pareto. FastGraph occupies the exact / fast region; approximate methods are only competitive when lower recall is acceptable.

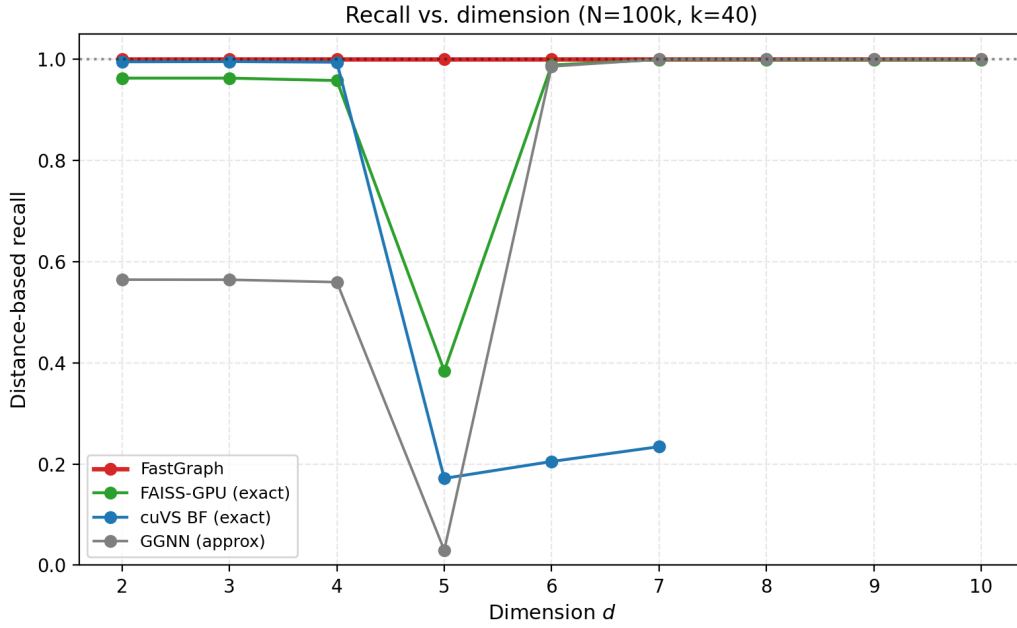


Figure 10: Distance-based recall versus dimension at the best-quality parameter setting ($N=10^5$, $k=40$). FastGraph holds recall = 1.0 across every dimension; CAGRA and GGNN degrade above $d=3$ and neither reaches 99%.

5.8. Dynamic graph construction in GNNs

GravNet-style layers [2] construct a fresh kNN graph in a learned latent space at every forward pass, so the kNN substrate is on the gradient path of every training step. FastGraph integrates into this setting through GravNetOp, a

PyTorch module that wraps coordinate transformation, kNN graph construction (via the PCA-subspace primitive of Section 3), and message passing into a single autograd-compatible operation. Gradients flow through the selected distances and the input coordinates; the discrete neighbor indices and the per-event projection V_k are held fixed across the forward-backward pair so that the gradients are deterministic within a step. Object condensation [3], which is the loss used together with GravNet for HGCAL reconstruction, is supported through a separate GPU-resident helper kernel described in Appendix Appendix A.

6. Limitations

What no longer applies.. The $d_{\max}=5$ binning cap inherited from axis-aligned cell-list implementations is removed by the PCA-subspace formulation: the projected grid is n_{bins}^k in the binning subspace size k , independent of the input dimensionality d . Per-column scale heterogeneity is also absorbed into the variance ordering of the principal components, so the binning structure no longer has to be told which input axes are large.

Data distribution still matters.. The PCA contribution’s effectiveness is a function of how much pairwise-distance variance the leading components actually capture. On strongly anisotropic data—HGCAL recHits, anything with a few dominant axes—FastGraph’s PCA-subspace binning is dramatically faster than axis-aligned cell lists (Section 5.5, geometric speedups in Section 5.1). On isotropic data, the projection is essentially a unitary rotation and FastGraph degrades gracefully toward ordinary cell-list behavior; in that regime FastGraph wins against dense GPU brute force only while the cell-list pruning advantage outweighs the constant factor of an optimized BLAS *gemm*. Section 5.3 documents this on Gaussian $\mathcal{N}(0, I_d)$ inputs and is the reason we do not claim FastGraph is the fastest GPU exact kNN on arbitrary distributions; we claim it is the fastest GPU exact kNN in the operating regime where the leading principal components carry real structural information, which is the regime modern learned-latent dynamic graphs are designed to live in.

What still applies.. PCA estimation itself adds an $O(Nd^2)$ term that is amortised over the kNN query in the regimes we benchmark, but at very small N the projection cost becomes a non-negligible fraction of the total wall-clock; for tiny per-event point counts an axis-aligned variant remains preferable. The current implementation targets a single GPU; multi-GPU or distributed kNN graph construction is not supported. The discrete neighbor indices are not differentiable; FastGraph’s autograd contract is over the continuous distances and coordinates only, matching every other hard-selection kNN layer. Among the GPU baselines, approximate methods remain faster than FastGraph at default low-recall settings; FastGraph is the right choice only when exact graph topology is required.

Why exactness matters for the downstream GNN.. The case for an exact kNN backend over a recall-tuned approximate one is not a raw-recall argument but a sensitivity-of-downstream argument. Adversarial-robustness studies of graph neural networks show that node predictions can shift substantially under small, structure-preserving edge perturbations to the input graph [29]; the same literature also documents that systematic reweighting of the neighbourhood—e.g., replacing direct neighbours with a diffusion-smoothed neighbour set—measurably alters downstream accuracy [30]. The neighbour-set mismatches injected by an approximate kNN are exactly this kind of perturbation, distributed non-uniformly across the event and dependent on the build-side index hyperparameters. We do not claim the HGCAL reconstruction targets are pathologically sensitive to approximate-kNN edge noise—that would require a dedicated model study we have not done—but the safer default for a physics application that demands reproducible, systematics-controlled inference is the exact graph, particularly when an exact backend is competitive on wall-clock in the relevant operating regime (Section 5.1).

CAGRA on heterogeneous data.. CAGRA’s default IVF-PQ build path fails on HGCAL data at $d \geq 5$ (Section 4); the NN-Descent fallback succeeds across the full dimensional sweep but is 17.36× slower than FastGraph at the $d=8$ reference cell. We report the NN-Descent numbers throughout. This is a finding about CAGRA on heterogeneous-scale features, not a generic CAGRA limitation; practitioners deploying CAGRA on similarly heterogeneous physics datasets should be aware of the failure mode and the appropriate build-path mitigation.

7. Data and Code Availability Statement

The FastGraph library, including the PCA-subspace extension described in this paper, is released at <https://github.com/AgarwalAarush/FastGraphCompute> (fork of the upstream library at <https://github.com/jkiesele/FastGraphCompute>). The exact source used to collect every headline timing in this paper is the paper-release branch at tag v1.1-paper (commit 4511df5), built against the CUDA 12.1 toolchain. v1.1-paper adds two memory-only optimisations to the autograd path (conditional `save_for_backward` and a fused index-replace+scatter kernel) on top of v1.0-paper (011d295); outputs are byte-identical and timings agree within run-to-run variance, so all reported timing numbers are unchanged. See Section 5.4. The released kernel surface covers binning subspace size $k \in \{2, 3, 4, 5\}$; the $k=6, 7$ kernels exist as an internal ablation (Section 5.5) and are intentionally not published, as $k=7$ hits a per-block register limit and is unsafe to ship. The benchmarking harness, queue files, calibration scripts, and the per-backend timing CSVs used to produce every figure in this paper live in a companion repository, <https://github.com/AgarwalAarush/fgc-performance>. The CLOVER head-to-head used a fork of <https://github.com/ampslab/clover-knn> at <https://github.com/AgarwalAarush/clover-knn> with local source modifications (`k_values` aligned to the paper’s $k=40$, the hardcoded synthetic-N template chain in `main()` disabled in favor of the mesh path, and the build retargeted to `sm_80` for A100).

The detector-derived benchmarks use CMS HGCAL simulation data (recHit feature vectors) accessed via an internal collaboration dataset of 1.16 billion calorimeter hits with up to ten spatial and energy features per hit. The data itself is internal to the CMS collaboration; synthetic Gaussian and uniform datasets used in Sections 5.3 and 5.6 are generated on the fly from fixed random seeds and are fully reproducible from the released code. CMS collaboration members can reproduce the HGCAL results directly; external parties should contact the corresponding author.

All benchmark timings in this paper come from a single installed binary of FastGraph (the canonical `fgc-fast` environment build, CUDA 12.1 toolchain). A separate development build under CUDA 13.2 shows a $\sim 50\%$ uniform slowdown across both PCA and axis-aligned paths that we have isolated to the toolchain transition rather than the source—this is a known regression in our build environment and does not affect the relative ordering of any result reported here.

8. Conclusion

We have introduced FastGraph, a PCA-subspace binned exact k -nearest-neighbor graph construction primitive for GPU geometric deep learning. The PCA-subspace formulation extends classical cell-list k NN past the small-dimensional cap that previously limited axis-aligned implementations, while a contraction argument ($V_k V_k^\top \leq I_d$) preserves exactness in the original input space. On the HGCAL recHit workloads that motivate this paper, across $d=2\text{--}10$ at the headline $N=5\text{ M}$, $k=40$ operating point, FastGraph is the fastest *exact* GPU k NN method at every tested dimension: geomean speedups of $10.1\times$ over FAISS-GPU IndexFlatL2, $4.8\times$ over cuVS brute force (the strongest exact GPU baseline), and $4.3\times$ over the approximate CAGRA-NN-Descent build—which is itself the only CAGRA path that runs at $d \geq 5$ on heterogeneous-scale detector data. The ablation in Section 5.5 isolates the PCA rotation rather than incidental implementation differences as the load-bearing contribution: comparing each variant at its own optimum gives a within-paper ratio of $26.6\times$ between PCA-subspace and axis-aligned binning at $d=8$, $N=5\text{ M}$, $k=40$.

This paper does not claim that FastGraph is the fastest GPU k NN on every distribution or every dimensionality. Two boundaries of the claim are documented. First, the deployment regime for the dynamic graph layers FastGraph targets is the $d=4\text{--}10$ learned latent space used by GravNet, ParticleNet, and HGCAL multi-particle reconstruction; the 3D-only spatial-acceleration methods (CLOVER, LBVH, RT-cores) are unavailable in the $d \geq 4$ regime by construction, and on $d=3$ HGCAL recHits the CLOVER spatio-graph index is roughly two orders of magnitude faster than FastGraph (Section 5.6). The $d=3$ detector-data case does not overlap with the regime FastGraph is designed for, but we report it honestly. Second, on *isotropic* data the PCA projection has no structure to exploit and FastGraph degrades toward ordinary cell-list behavior; for that case the relative ordering against dense BLAS brute force is determined by the cell-list pruning advantage alone, not by the PCA contribution. The HGCAL gains are an anisotropic-data result, and we expect PCA-subspace binning to be a generally useful substrate for dynamic graph layers in scientific GNNs whose latent spaces have similar spectral structure.

References

- [1] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, J. Leskovec, Graph convolutional neural networks for web-scale recommender systems, in: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2018, pp. 974–983.
- [2] S. R. Qasim, J. Kieseler, Y. Iiyama, M. Pierini, Learning representations of irregular particle-detector geometry with distance-weighted graph networks, *The European Physical Journal C* 79 (7) (2019) 608.
- [3] J. Kieseler, Object condensation: One-stage grid-free multi-object reconstruction in physics detectors, graphs, and images, *The European Physical Journal C* 80 (9) (2020) 886.
- [4] S. R. Qasim, et al., Multi-particle reconstruction in the high granularity calorimeter using object condensation and graph neural networks, *arXiv preprint arXiv:2106.01832* (2021).
- [5] S. Bhattacharya, et al., GNN-based end-to-end reconstruction in the CMS phase-2 high-granularity calorimeter, *arXiv preprint arXiv:2203.01189* (2022).
- [6] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, J. M. Solomon, Dynamic graph CNN for learning on point clouds, *ACM Transactions on Graphics* 38 (5) (2019) 1–12.
- [7] H. Qu, L. Gouskos, ParticleNet: Jet tagging via particle clouds, *Physical Review D* 101 (5) (2020) 056019.
- [8] K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft, When is “nearest neighbor” meaningful?, in: *International Conference on Database Theory (ICDT)*, Vol. 1540 of *Lecture Notes in Computer Science*, Springer, 1999, pp. 217–235.
- [9] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, *IEEE Transactions on Big Data* 7 (3) (2021) 535–547.
- [10] J. Feydy, J. Glaunès, B. Charlier, M. Bronstein, Fast geometric learning with symbolic matrices, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 14448–14462.
- [11] A. Dashti, I. Komarov, R. M. D’Souza, Efficient computation of k-nearest neighbour graphs for large high-dimensional data sets on GPU clusters, *PLOS ONE* 8 (9) (2013) e74113.
- [12] I. Komarov, A. Dashti, R. M. D’Souza, Fast k-NNG construction with GPU-based quick multi-select, *PLOS ONE* 9 (5) (2014) e92409.
- [13] R. F. Sproull, Refinements to nearest-neighbor searching in k-dimensional trees, in: *Algorithmica*, Vol. 6, 1991, pp. 579–589.
- [14] Y. Zhu, RTNN: Accelerating neighbor search using hardware ray tracing, in: *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, ACM, 2022, pp. 76–89.
- [15] V. Nagarajan, D. Mandarapu, M. Kulkarni, RT-kNNS unbound: Using RT cores to accelerate unrestricted neighbor search, in: *Proceedings of the 37th ACM International Conference on Supercomputing (ICS)*, ACM, 2023.
- [16] D. K. Mandarapu, V. Nagarajan, A. Pelenitsyn, M. Kulkarni, Arkade: k-nearest neighbor search with non-Euclidean distances using GPU ray tracing, in: *Proceedings of the 38th ACM International Conference on Supercomputing (ICS)*, ACM, 2024. doi:10.1145/3650200.3656601.
- [17] J. Jakob, M. Guthe, Optimizing LBVH-construction and hierarchy-traversal to accelerate kNN queries on point clouds using the GPU, *Computer Graphics Forum* 40 (1) (2021) 124–137.
- [18] V. Kamel, H. Yan, S. Chester, CLOVER: A GPU-native, spatio-graph-based approach to exact kNN, in: *Proceedings of the 39th ACM International Conference on Supercomputing (ICS)*, ACM, 2025. doi:10.1145/3721145.3730415.
- [19] W. Zhao, S. Zhang, Y. Ding, Z. Tan, J. Liu, D. Zhan, SONG: Approximate nearest neighbor search on GPU, in: *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, IEEE, 2020, pp. 1033–1044.
- [20] F. Groh, L. Ruppert, P. Wieschollek, H. P. A. Lensch, GGNN: Graph-based GPU nearest neighbor search, *IEEE Transactions on Big Data* 9 (1) (2023) 267–279.
- [21] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, Y. Wang, CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs, in: *2023 IEEE International Conference on Big Data (BigData)*, IEEE, 2023, pp. 33–42.
- [22] H. Wang, W.-L. Zhao, X. Zeng, Large-scale approximate k-NN graph construction on GPU, *arXiv preprint arXiv:2103.15386* (2021).
- [23] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4) (2020) 824–836.
- [24] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, S. Kumar, Accelerating large-scale inference with anisotropic vector quantization, in: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 3887–3896.
- [25] E. Bernhardsson, Annoy: Approximate nearest neighbors in C++/Python optimized for memory usage and loading/saving to disk, Open-source software library, Spotify, version 1.17.3 (2013).
URL <https://github.com/spotify/annoy>
- [26] X. Ju, et al., Performance of a geometric deep learning pipeline for HL-LHC particle tracking, *The European Physical Journal C* 81 (10) (2021) 876.
- [27] A. Lazar, et al., Accelerating the exa.TrkX pipeline, *Computing and Software for Big Science* 6 (1) (2022) 1–9.
- [28] N. Choma, et al., Track seeding and labelling with embedded-space graph neural networks, *arXiv preprint arXiv:2007.00149* (2020).
- [29] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, ACM, 2018, pp. 2847–2856. doi:10.1145/3219819.3220078.
- [30] J. Klicpera, S. Weißenberger, S. Günnemann, Diffusion improves graph learning, in: *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, arXiv:1911.05485.

Appendix A. Object Condensation Helper — Implementation Details

In addition to the kNN primitive, the library provides a GPU-resident CUDA kernel for the object condensation loss [3], widely used to cluster sensor-hit point clouds into particle candidates in physics reconstruction. Its training loop requires two dense auxiliary index matrices to be recomputed at every forward pass:

1. The *association matrix* $M \in \mathbb{Z}^{n_{\text{unique}} \times n_{\text{max } uq}}$, whose k -th row lists the vertex indices assigned to the k -th object candidate.
2. The optional *complement matrix* $M_- \in \mathbb{Z}^{n_{\text{unique}} \times n_{\text{max } rs}}$, listing vertices *not* assigned to the candidate; required only when the repulsive term m_- is included in the loss.

Both matrices must be recomputed every forward pass because the predicted associations and the ragged batch layout change from iteration to iteration. The CUDA kernel parallelizes over unique object candidates, assigning one thread per candidate, eliminating atomic operations and thread synchronization and ensuring collision-free writes to the output matrices at the cost of $O(n_{\text{unique}})$ launched threads per forward pass.

Algorithm 2: CUDA Object Condensation Helper (oc_helper): one thread per candidate.

```

Input: asso_idx[n_v], unique_idx[n_u], unique_rs_asso[n_u], rs[n_s+1], n_v, n_u, n_max_uq, n_max_rs, flag
        calc_m_not
Output: M[n_u][n_max_uq], M_-[n_u][n_max_rs]
1 k ← blockIdx.x · blockDim.x + threadIdx.x
2 if k ≥ n_u then
3   return
4 uq ← unique_idx[k]; split ← unique_rs_asso[k]; start ← rs[split]; end ← min(rs[split+1], n_v)
5 if end - start > n_max_rs then
6   end ← start + n_max_rs
7 fill ← 0
8 for i ← start to end - 1 do
9   if asso_idx[i] = uq then
10    M[fill][k] ← i; fill ← fill + 1
11    if fill ≥ n_max_uq then
12      break
13 while fill < n_max_uq do
14   M[fill][k] ← -1; fill ← fill + 1
15 if calc_m_not then
16   fill ← 0
17   for i ← start to end - 1 do
18     if asso_idx[i] ≠ uq then
19       M_-[fill][k] ← i; fill ← fill + 1
20       if fill ≥ n_max_rs then
21         break
22 while fill < n_max_rs do
23   M_-[fill][k] ← -1; fill ← fill + 1

```
